

Linalg-to-NKI: An MLIR Pipeline that Lowers Triton Kernels to the AWS Trainium Tensor Engine

Devanshu Mantri
University of Utah
u1551687@utah.edu

Venkatesh Bellale
University of Utah
u1529208@utah.edu

Abstract

Triton is already the dominant user-facing language for custom deep-learning kernels: the major model frameworks ship Triton kernels, and the community of developers who can read and write Triton dwarfs the community who can write AWS Trainium’s Neuron Kernel Interface (NKI). NKI, by contrast, is a newer Python-embedded ISA specific to the Trainium tensor engine. Any Triton program that runs on Trainium at all today does so through a hand-written NKI re-implementation. The disparity in reach between the two languages is the core motivation of this project: if a Triton kernel can be compiled automatically to NKI, the existing Triton ecosystem is, in one step, available on Trainium.

We built a prototype for this compiler workflow. Our contribution is a small MLIR dialect and six passes that recognise the IR shape produced by `triton-shared`’s lowering of a Triton `matmul` and rewrite it into our dialect, plus an `mlir-translate`-style tool that emits `@nki.jit` Python. The end-to-end pipeline handles a canonical Triton `matmul`, a fused `matmul+ReLU`, and a vector-add, producing kernels that match `torch.matmul` across an 8-configuration $\times$ 10-shape block sweep and that are structurally comparable to the `nki-samples` reference kernels.

Keywords

MLIR, Triton, AWS Trainium, NKI, tensor compilers, dialect design, pattern rewriting

1 Introduction

Hardware accelerators for deep learning increasingly expose non-uniform, partitioned memory hierarchies. AWS Trainium’s second-generation Neuron Core (NC-v2) [1] has a 128-lane systolic tensor engine, a per-lane scratch memory (SBUF), and a separate PSUM accumulator bank that sits between the engine and SBUF. The official programming model, NKI (the Neuron Kernel Interface), is a Python-embedded DSL in which the programmer writes loops over explicit tile shapes and selects one of three buffer kinds—SBUF, PSUM, or shared HBM—for every tensor.

Triton [3], which originated on NVIDIA GPUs, has since grown into the dominant user-facing language for custom deep-learning kernels, and its IR (TTIR) is hardware-independent

enough that an open-source project, `triton-shared` [4], already lowers TTIR to upstream MLIR `linalg`. What is missing, and what this project fills in, is the leg from `linalg` to NKI—the piece that would make the large body of existing Triton kernels available on Trainium without rewriting each one by hand in the NKI Python DSL.

Contributions. We contribute:

- (1) A small MLIR dialect whose five operations each correspond 1:1 to a specific NKI Python idiom (§4).
- (2) Six MLIR passes that together recognise the exact IR pattern produced by `triton-shared`’s lowering of a Triton `matmul` (with optional pointwise epilogue) and rewrite it into our dialect (§5).
- (3) A pattern-matching Python emitter that walks the lowered IR and prints a `@nki.jit`-decorated Python kernel (§6).
- (4) An end-to-end driver, an 8-configuration block sweep, and an on-hardware correctness harness covering aligned, ragged, and degenerate shapes (§7).

The prototype supports Triton programs that fit the skeleton *grid-over-tiles*, *K-loop with PSUM accumulator*, *optional pointwise epilogue*. We explicitly do not attempt a general TTIR-to-NKI compiler; the goal is a compilation leg that is narrow enough to be verifiable and broad enough to be useful.

2 Background

2.1 NC-v2 tensor engine and the NKI programming model

A Trainium NC-v2 core exposes three memory tiers [1]: global HBM (accessed via explicit DMA); a 128-partition scratchpad (`n1.sbuf`) whose partition axis is physically mapped to the 128 lanes of the tensor engine; and a PSUM accumulator bank (`n1.psum`) holding in-flight `matmul` partial sums. The tensor engine itself computes $\text{dst} += \text{stationary}^T \cdot \text{moving}$ for a $(128, M)$ stationary operand and a $(128, N)$ moving operand, where $M \leq 128$ and $N \leq 512$.

NKI exposes these primitives as Python function calls: `n1.ndarray(..., buffer=n1.psum)` allocates a PSUM tile, `n1.load / n1.store` issue DMAs, and `nisa.nc_matmul` issues a single tensor-engine instruction. A textbook `matmul`

```

Triton @triton.jit
↓ Triton front-end
TTIR (tt.* ops)
↓ triton-shared-opt -triton-to-linalg
↓ -linalg-specialize-generic-ops
specialized linalg IR
↓ -nki-canonicalize-pid-loops
↓ -linalg-to-nki
↓ -nki-fuse-dma, -nki-fuse-store
↓ -nki-fuse-activation (opt.)
↓ -nki-fold-psum-init
lowered nki.* IR
↓ linalg-to-nki-translate
NKI Python (@nki.jit)

```

Figure 1: End-to-end compilation pipeline. Steps above the first horizontal rule are upstream; steps below are contributions of this project.

therefore has a three-level loop nest (M-tiles, N-tiles, K-tiles inside), with the K-loop folding partials into a single PSUM tile allocated *outside* the K-loop to preserve the accumulator.

2.2 Triton and triton-shared

A Triton matmul kernel declares BLOCK sizes, computes two-dimensional tile offsets from `tl.program_id`, loads tiles with masks, updates an FP32 accumulator via `tl.dot`, and stores the result with a two-dimensional mask. The Triton front-end compiles this to *Triton IR* (TTIR), a dialect of MLIR.

`triton-shared` then lowers TTIR into upstream MLIR dialects (`linalg`, `memref`, `scf`, `tensor`, `bufferization`). After the accompanying `-linalg-specialize-generic-ops` pass, the matmul body emerges as a destination-passing `linalg.matmul + linalg.add` pair, the K-loop as an `scf.for` carrying an `iter_args` accumulator seeded by `tensor.empty + linalg.fill 0.0`, and every masked tile load/store as a `reinterpret_cast + subview + memref.copy + to_tensor` chain. Our passes consume exactly this shape.

2.3 MLIR out-of-tree dialects

MLIR’s out-of-tree pattern [5] lets a third party define a new dialect and a set of passes without touching the upstream tree. We follow the standard layout: a `.td` file describing ops and passes, a `CMakeLists.txt` that links against the MLIR C++ libraries, and a pair of driver binaries (`linalg-to-nki-opt`, `linalg-to-nki-translate`) that mirror the upstream `mlir-opt` / `mlir-translate` convention.

3 System Overview

Figure 1 sketches the full pipeline. The user writes a Triton kernel; `generate_kernels.py` orchestrates the entire chain. The first two steps (Triton $\rightarrow$ TTIR $\rightarrow$ specialized linalg) use only upstream tooling. The middle four to six steps are

our MLIR pass pipeline, dispatched by `linalg-to-nki-opt`. The final translation step is our `linalg-to-nki-translate` binary, which reads the lowered IR and emits Python.

The split into six passes (rather than one monolithic pattern) is deliberate. Each pass targets a single IR pattern: a pre-condition on what it expects to see and a post-condition on what it leaves behind. The passes are further ordered so that each one sees a strictly simpler IR than the one before it, which keeps the later patterns local and substantially easier to write.

4 The nki Dialect

The `nki` dialect holds five operations; each one has a dedicated Python emission rule. The dialect is deliberately small—it is not designed for general-purpose optimisation but as a pattern-matching target that is strictly simpler to translate than raw `linalg`.

`nki.nc_matmul`. Models one tensor-engine instruction: `result = acc + lhs * rhs` on a single tile. Takes an `lhs` ($M \times K$), an `rhs` ($K \times N$), and a prior accumulator `acc` ($M \times N$), and produces the updated accumulator. The op is `Pure`, enforces matching element types across all operands and matching shapes on `acc/result`, and carries a verifier that checks the tile-shape contract.

`nki.psum_alloc`. Zero-arity op that models `nl.ndarray((M,N), dtype=..., buffer=nl.psum)`. The `-nki-fold-psum-init` pass inserts it in place of the `tensor.empty + linalg.fill 0.0` pair that seeds the K-loop accumulator.

`nki.dma_copy` / `nki.dma_store`. Load and store sides of a semantic DMA. Each carries an unranked `memref` base (the global buffer), a linearised element offset, and one index stride per tile dimension, mirroring the triple already carried by `memref.reinterpret_cast`. `dma_store` additionally carries a string-valued activation attribute (default “none”) that the activation-fusion pass populates.

The store op is intentionally *not* `Pure`; the greedy pattern driver would otherwise DCE it together with its now-dead upstream `memref` chain.

`nki.tensor_tensor`. A binary elementwise op carrying a string-valued mnemonic attribute (“add”, “sub”, ...). This is the target of `linalg-to-nki`’s secondary pattern, which handles the vector pointwise case: a `linalg.add` whose operands are not produced by a `linalg.matmul`.

Listing 1 shows a representative fragment of the IR after the full pass pipeline has run; the translation to NKI Python reads almost directly off this form.

Listing 1: Fragment of the IR after the full pass pipeline has run. The `nki.*` ops each map to one line of Python.

```

1 %psum = nki.psum_alloc : tensor<128x512xf32>
2 scf.for %m = %c0 to %cnumM step %c1 {
3   scf.for %n = %c0 to %cnumN step %c1 {
4     %acc = scf.for %k = %c0 to %cnumK step %c1
5       iter_args(%a = %psum) -> tensor<128x512xf32> {
6         %A = nki.dma_copy %pa[%oA] strides [%sA0, %sA1]
7           : memref<*xf32> to tensor<128x128xf32>
8         %B = nki.dma_copy %pb[%oB] strides [%sB0, %sB1]
9           : memref<*xf32> to tensor<128x512xf32>
10        %r = nki.nc_matmul %A, %B, %a : (...) -> ...
11        scf.yield %r : tensor<128x512xf32>
12      }
13      nki.dma_store %acc into %pc[%oC]
14        strides [%sC0, %sC1] {activation = "none"}
15        : tensor<128x512xf32> into memref<*xf32>
16    }
17  }

```

5 Compiler Passes

This section walks through the six passes in dispatch order. Each pass is implemented as a set of OpRewrite patterns driven by MLIR’s `applyPatternsAndFoldGreedily` and lives in its own file under `Transforms/`.

5.1 -nki-canonicalize-pid-loops

Triton kernels receive a flat `pid = tl.program_id(0)` and recover 2D tile coordinates via `pid_m = pid // num_pid_n`, `pid_n = pid % num_pid_n`. After `triton-shared`, this becomes a sequence of `arith.divsi/arith.remsi` on a function argument, with the tile offsets computed from a pair of constants that are *not* syntactically `num_pid_n` but rather `BLOCK_M` and `BLOCK_N`.

The pass recognises the exact constant-pattern

$$\lfloor N / \text{BLOCK_N} \rfloor = \text{num_pid_n}$$

via a match on the `addi + divsi` pair whose right-hand side is the $(\text{BLOCK_N} - 1)$ constant, then wraps the entire function body in two `scf.for` loops over $(\text{pid}_m, \text{pid}_n)$, replaces the `divsi / remsi` results with the new induction variables, removes the `program_id` function argument, and rewires the callers. The loop bound for the `m` dimension is reconstructed from the `M` argument and the recovered `BLOCK_M`; that argument is located as the `i32` value used in the load-side `minsi` bounds-check chain. The ordering constraint is important: this pass must run *before* the DMA-fusion pass, which erases that `minsi` chain.

Recovering the loop nest structurally (rather than leaving the divisions in place) makes every downstream pass simpler,

because they can each ignore the `pid` plumbing and focus on the tile-level computation.

5.2 -linalg-to-nki

This pass holds the two primary rewrites. The *matmul accumulator* pattern fires on any `linalg.add` whose inputs are a prior accumulator and the result of a `linalg.matmul` whose output-init is a `linalg.fill` of an `arith.constant` zero. All four ops—the add, the `matmul`, the fill, and the now-dead `tensor.empty`—collapse into a single `nki.nc_matmul`.

It is worth noting why `triton-shared` emits this as a *separate* `linalg.matmul` plus `linalg.add` rather than feeding the running accumulator directly into the `matmul`’s outs operand. `triton-shared` is a target-agnostic lowering: different accelerators run the product and the accumulation at different precisions (fp16 or bf16 multiply with an fp32 accumulate is common), so keeping the multiply and the add as two explicit ops preserves each backend’s freedom to pick its own element type for each half.

Our dialect does *not* currently exploit that freedom. The `nki.nc_matmul` op constrains all four of lhs, rhs, acc, and result to share one element type, and the pipeline is fp32 end-to-end. Extending the op to a two-type form (one type for the inputs, one for the accumulator) to match the tensor engine’s native fp16/bf16-in, fp32-accumulate mode is the most obvious next step (§9).

A second detail is that we keep an acc operand on our `nc_matmul` even though the Python emitter does not pass it to `nisa.nc_matmul`. The emitted call is an in-place update of the PSUM tile,

```
res_psum[...] += nisa.nc_matmul(...),
```

so accumulation happens through PSUM’s `+=` semantics rather than through a `dst=` argument on the ISA call. We still keep `acc` in our IR because the later `fold-psum-init` pass (§5.5) anchors on it: it walks the `acc` value back through the surrounding `scf.for`’s `iter_args` to the `linalg.fill` that seeds the accumulator, and only rewrites when that chain is well-formed. Dropping `acc` would leave no direct handle for that traversal.

The *pointwise* pattern fires on any `linalg.add` whose operands are *not* produced by a `linalg.matmul`—i.e. the ordinary vector-add case—and rewrites it to `nki.tensor_tensor "add"`. The two patterns are disjoint by construction: their match conditions are mutually exclusive.

A subtlety is that `triton-shared` emits the `matmul` body in destination-passing style, so the `linalg.add`’s outs operand is the same SSA value as one of its inputs. The pattern must dereference *which* input is the accumulator by asking which input is also the output-init, not by operand position, because `triton-shared` is free to pick either ordering.

5.3 -nki-fuse-dma

A single masked tile load in Triton arrives from `triton-shared` as a five-step memref dance: a reinterpret-cast that attaches a base offset and per-axis strides to the global pointer; a fresh alloc for the tile buffer; two mirrored subviews that clamp the source and destination to the in-bounds sub-rectangle; a copy that performs the transfer; and a final bufferize-to-tensor that hands the tile back to the rest of the IR in tensor form. Listing 2 shows the exact shape.

Listing 2: Triton-shared load-side chain (before -nki-fuse-dma).

```
1 %rc = memref.reinterpret_cast %src to
2   offset: [%o], sizes: [M, N], strides: [%s0, %s1]
3 %buf = memref.alloc() : memref<MxNxT>
4 %ssrc = memref.subview %rc [0,0][%vr, N][1,1]
5 %sdst = memref.subview %buf [0,0][%vr, N][1,1]
6 memref.copy %ssrc, %sdst
7 %tile = bufferization.to_tensor %buf restrict writable
```

The pass collapses this whole chain into a single semantic load,

```
%tile = nki.dma_copy %src [%o] strides [...],
```

and then explicitly erases the five ops it replaced. Erasure must be explicit: the `memref.copy`, the `memref.alloc`, and the `reinterpret-cast/subview` pair all have memory side effects, so MLIR’s greedy pattern driver will not dead-code-eliminate them on its own.

The one thing the rewrite *drops* is the dynamic clamp `%vr`, which comes from Triton’s `minsi` bounds-check on the row count. Dropping it is safe because the eventual NKI Python already masks every element individually against the dynamic problem size M/N via the `mask=` argument to `n1.load`, so the same bounds-check happens later, one level down. Keeping `%vr` in the IR would force the emitter to either plumb it through the translator or print dead code that does nothing. Neither is worth the structural information it would preserve.

Note that `nki.dma_copy` is an *IR-level* op, not a Python call. The Python emitter (§6) is what lowers each `nki.dma_copy` in the IR to an `n1.load(...)` call with a computed `mask=` argument; that is why the generated `.py` files show `n1.load` rather than the dialect op name.

5.4 -nki-fuse-store

The store side is symmetric. `triton-shared` produces a reinterpret-cast on the output pointer, an extract-slice that takes the in-bounds sub-rectangle of the tile, a mirrored subview on the destination, and a `bufferization.materialize_in_destination` that writes the slice out (Listing 3).

Listing 3: Triton-shared store-side chain (before -nki-fuse-store).

```
1 %rc = memref.reinterpret_cast %dst to
2   offset: [%o], sizes: [M, N], strides: [%s0, %s1]
3 %slice = tensor.extract_slice %tile[0,0][%vr,%vc][1,1]
4 %sdst = memref.subview %rc [0,0][%vr, %vc][1,1]
5 bufferization.materialize_in_destination
6   %slice in writable %sdst
```

The pass collapses the chain into a single `nki.dma_store` that carries the same base, offset, and strides, and erases the rest. As on the load side, the dynamic extents `%vr/%vc` are deliberately discarded: the emitter masks the final `n1.store` against M/N at the element level, so the bounds check is preserved one abstraction layer lower rather than kept redundantly in the IR.

5.5 -nki-fold-psum-init

After the `matmul` pattern has fired, the K-loop looks like

```
1 %0 = tensor.empty()
2 %1 = linalg.fill ins(%cst : f32) outs(%0)
3 %r = scf.for ... iter_args(%acc = %1) -> tensor<MxNxf32>
4   {
5     ...
6     %m = nki.nc_matmul %A, %B, %acc
7     scf.yield %m
8   }
```

The fill is the PSUM-zero seed; we rewrite the `linalg.fill` to a `nki.psum_alloc` and erase the now-dead `tensor.empty`. The pass is narrow: it only fires when the fill’s single use is an `scf.for iter_args` and the corresponding loop argument flows into an `nki.nc_matmul acc` operand. Firing more broadly would be unsound: any fill-of-zero in the function would be rewritten to a PSUM allocation, but PSUM is a separate physical memory with its own liveness rules.

The downstream Python emitter inspects the `nki.psum_alloc`-fed `iter_args` chain to decide where to place the `n1.zeros(..., buffer=n1.psum)` call—always *outside* the K-loop, so that the PSUM accumulator persists across K-iterations.

5.6 -nki-fuse-activation

This pass handles the fused-epilogue case. A Triton kernel that ends with `tl.maximum(acc, 0.0)` lowers, after `triton-shared`, to an elementwise `linalg.generic` whose body is a single `arith.maxnumf` against a second zero-filled tensor. The pass:

- (1) recognises the generic body—shapes supported include unary math ops (`math.exp`, `log`, `sqrt`, `rsqrt`, `absf`, `tanh`, `erf`, `sin`, `cos`, `tan`) and binary ops with a zero-filled sibling (`arith.maxnumf` $\rightarrow$ `ReLU`);

- (2) verifies that the only consumer of the generic is the final `dma_store`; and
- (3) rewrites by deleting the generic, rewiring the store to take the pre-activation accumulator, and setting the store’s activation attribute to the matching NKI activation name.

A companion pass, `-nki-check-unsupported-elementwise`, walks the post-fusion IR looking for any *surviving* element-wise `linalg.generic/linalg.max` that the emitter cannot handle. For each, it emits a readable diagnostic and a note listing the supported activations. This turns an otherwise silent crash in the translator into a proper compiler error.

6 The Python Emitter

The emitter (`linalg-to-nki-translate`) is an `mlir-translate-style` tool, not a pass. It reads the fully lowered IR, identifies the single `func.func`, classifies it as either a matmul or a pointwise kernel, and walks the body printing Python.

Kernel classification. The emitter first walks the function top-to-bottom looking for an `nki.nc_matmul`. If one is found, the kernel is a matmul; the emitter then infers the tile sizes (M, N, K) from the `nc_matmul`’s operand types. If none is found, the emitter looks for an `nki.tensor_tensor` and infers the 1D block size from its result type. The two kinds share the same DMA and epilogue rules but emit structurally different headers (2D grid vs 1D flat loop).

Matmul emission. The emitter prints a header that declares `TILE_M=128, TILE_N=512, TILE_K=128` (the NC-v2 canonical sizes), regardless of the Triton BLOCK constants. This is safe because the full mask-every-load pipeline inside the emitted body never depends on the BLOCK boundary—it masks each element individually against the dynamic M, K, N function arguments. It means a single compile is sufficient; an upstream BLOCK sweep does not produce different Python.

The body prints three nested `nl.affine_range` loops that mirror the three `scf.fors` in the IR. The outer m and n loops use `ceil-div` bounds. Inside them, the emitter prints an `nl.zeros` allocated in PSUM that corresponds to the `nki.psum_alloc`, then the inner k loop. For each `nki.dma_copy` inside the K -loop it prints an `nl.load` call with an `mgrid` index, an explicit SBUF destination, and a 2D mask that is the element-wise AND of per-axis range checks against K, M , and N . The `nki.nc_matmul` prints as a call to `nisa.nc_matmul` with `stationary=` and `moving=` kwargs, plus an in-place `+=` update of the PSUM tile. Finally, the `nki.dma_store` prints as either a `nisa.tensor_copy` (when `activation="none"`) or a call to `nisa.activation` with `op=nl.<name>` (for any other attribute), followed by an `nl.store` with the full 2D mask.

Pointwise emission. For a vector-add kernel, the emitter prints a single `nl.affine_range` over the block count, one `nl.load` for each operand, an `nl.add` (or the corresponding `nisa.tensor_tensor` mnemonic), and one `nl.store`. No PSUM is involved.

Structural fidelity. The emitted kernels are structurally identical to the `nki-samples` hand-tuned references for their respective operations, with one deliberate difference: every load and store in the emitted code carries a mask, whereas the `nki-samples` matmul requires M, K , and N to be exact multiples of 128, 128, and 512 respectively. The generated kernel therefore supports arbitrary shapes (including $1 \times 1 \times 1$), which was a hard requirement for us—Triton’s masking semantics mean that losing it would change observable behaviour.

7 Evaluation

7.1 Research questions

We frame the evaluation around three questions.

- RQ1.** Does the full pipeline run end-to-end on a canonical Triton matmul, a matmul+ReLU, and a vector-add, and produce syntactically valid `@nki.jit` Python?
- RQ2.** Are the emitted kernels numerically equivalent to `torch.matmul` (resp. a reference vector-add) across a spectrum of aligned, ragged, and degenerate shapes?
- RQ3.** How do the emitted kernels compare, structurally and (where available) in wall-clock time, to the `nki-samples` hand-tuned references?

7.2 Setup

All passes are developed against MLIR at a version synchronised with the `triton` submodule’s vendored LLVM. `triton-shared` is built in-tree alongside Triton. The pipeline driver (`generate_kernels.py`) runs on a standard Linux host; the only dependency is a working `triton` Python environment.

The numerical-correctness harness runs on a Trainium instance with `torch_xla`. Each shape is compared to `torch.matmul` on the XLA device at `atol=1e-4, rtol=1e-2` (fp32, matching `nki-samples` tolerances).

7.3 RQ1: end-to-end pipeline coverage

We compile three Triton programs through the full pipeline:

- (1) a textbook $M \times K \cdot K \times N$ matmul with BLOCK (128, 512, 128);
- (2) the same matmul with a fused `tl.maximum(acc, 0.0)` epilogue;
- (3) a 1D vector-add across a sweep of BLOCK sizes 128, 256, 512, 1024, and 2048.

All three compile cleanly through all four stages (Triton $\rightarrow$ TTIR $\rightarrow$ linalg $\rightarrow$ lowered `nki` IR $\rightarrow$ Python) without manual intervention, producing the emitted `.py` kernels for

Listing 4: Final emitted NKI kernel for the canonical Triton matmul, produced end-to-end by linalg-to-nki-translate. Every `nl.load` and `nl.store` carries a per-element mask against the dynamic M , K , N arguments; the inner `nisa.nc_matmul` issues one tensor-engine instruction per K -tile; the PSUM allocation is hoisted outside the K -loop to preserve the accumulator.

```

1  import neuronxcc.nki as nki
2  import neuronxcc.nki.isa as nisa
3  import neuronxcc.nki.language as nl
4
5
6  @nki.jit
7  def matmul_kernel_nki(lhsT, rhs):
8      TILE_M = 128
9      TILE_N = 512
10     TILE_K = 128
11
12     K, M = lhsT.shape
13     K_, N = rhs.shape
14     assert K == K_
15
16     C = nl.ndarray((M, N), dtype=lhsT.dtype, buffer=nl.shared_hbm)
17
18     for m in nl.affine_range((M + TILE_M - 1) // TILE_M):
19         for n in nl.affine_range((N + TILE_N - 1) // TILE_N):
20             res_psum = nl.zeros((TILE_M, TILE_N),
21                                 dtype=nl.float32, buffer=nl.psum)
22
23             for k in nl.affine_range((K + TILE_K - 1) // TILE_K):
24                 i_k, i_m = nl.mgrid[0:TILE_K, 0:TILE_M]
25                 mask_lhsT = (k*TILE_K + i_k < K) & (m*TILE_M + i_m < M)
26                 lhsT_tile = nl.zeros((TILE_K, TILE_M),
27                                     dtype=lhsT.dtype, buffer=nl.sbuf)
28                 lhsT_tile[i_k, i_m] = nl.load(
29                     lhsT[k*TILE_K + i_k, m*TILE_M + i_m],
30                     mask=mask_lhsT)
31
32                 i_k, i_n = nl.mgrid[0:TILE_K, 0:TILE_N]
33                 mask_rhs = (k*TILE_K + i_k < K) & (n*TILE_N + i_n < N)
34                 rhs_tile = nl.zeros((TILE_K, TILE_N),
35                                    dtype=rhs.dtype, buffer=nl.sbuf)
36                 rhs_tile[i_k, i_n] = nl.load(
37                     rhs[k*TILE_K + i_k, n*TILE_N + i_n],
38                     mask=mask_rhs)
39
40                 res_psum[...] += nisa.nc_matmul(
41                     stationary=lhsT_tile, moving=rhs_tile)
42
43                 i_m, i_n = nl.mgrid[0:TILE_M, 0:TILE_N]
44                 res_sbuf = nisa.tensor_copy(res_psum, dtype=lhsT.dtype)
45                 nl.store(C[m*TILE_M + i_m, n*TILE_N + i_n],
46                         value=res_sbuf,
47                         mask=(m*TILE_M + i_m < M) & (n*TILE_N + i_n < N))
48     return C

```

B_M	B_N	B_K	description
32	32	32	tiny cube (all < PE)
64	64	64	small cube
128	128	128	cube-128
128	512	128	canonical (matches emitter)
256	128	128	wide M
128	256	128	wide N (below canonical)
128	128	256	wide K
64	128	32	mixed small

Table 1: Triton BLOCK configurations covered by the sweep.

M	K	N	description
128	128	512	single tile, exact
256	128	512	2 M-tiles
128	128	1024	2 N-tiles
100	128	512	partial M
128	128	400	partial N
128	100	512	partial K
200	150	300	partial M, K, N (small)
500	250	700	partial M, K, N (mid)
1	1	1	degenerate $1 \times 1 \times 1$
64	1	64	degenerate $M \times 1 \times N$

Table 2: Shape cases used to cross-check every sweep kernel against torch.matmul.

each workload. The intermediate TTIR, specialized linalg, and lowered MLIR are persisted alongside each output for inspection.

Across the matmul+ReLU output, the -nki-fuse-activation pass successfully rewrites the linalg.generic + arith.maxnumf into a {activation = "relu"} attribute on the store. The Python emitter consequently prints nisa.activation(op=nl.relu, data=res_psum, dtype=...) in place of the nisa.tensor_copy that would otherwise appear, eliminating one full HBM round-trip (the unfused reference ref_matmul_relu_unfused in reference_kernels.py stores the matmul result to HBM and then reloads it for a standalone ReLU).

7.4 RQ2: numerical correctness over a block sweep

The real stress test is the *block sweep*. Recall that the emitter hardcodes TILE = (128, 512, 128) regardless of the upstream Triton BLOCK constexpr; we wanted to verify this is sound. generate_sweep_kernels.py drives the full pipeline on the eight BLOCK configurations in Table 1, producing eight independently compiled Python kernels.

aspect	nki-samples ref	generated kernel
M%128, K%128, N%512	required	not required
loop structure	3-deep affine	3-deep affine
PSUM placement	outside K-loop	outside K-loop
DMA per tile	nisa.dma_copy	nl.load (masked)
PSUM → SBUF	tensor_copy	tensor_copy / activation
final write	nisa.dma_copy	nl.store (masked)
ReLU epilogue	separate kernel	fused in activation=

Table 3: Structural comparison of the generated kernel against the nki-samples tutorial reference.

Each kernel is then run (on-device) against the ten shape cases in Table 2, covering tile-aligned, ragged, and degenerate regimes. Because the emitter is BLOCK-invariant, all eight kernels produce byte-identical Python and consequently pass identically; the sweep is nonetheless a useful regression harness, since a future emitter change could break the invariant silently.

Across the $8 \times 10 = 80$ cases the emitted kernels agree with torch.matmul within the stated tolerance, including the $1 \times 1 \times 1$ case (every load and store is masked, every out-of-range position is thrown away before the final accumulate). An analogous canonical suite (tests/canonical/test_matmul.py) cross-checks the single canonical kernel against the same shape family.

The ReLU kernel is covered by tests/canonical/test_matmul_relu.py on the same shape family, with the ground truth being $\max(0, A \cdot B)$ computed via PyTorch. It passes on the full suite.

The ten cases in Table 2 split into three regimes, each catching a distinct class of failure. The tile-aligned rows exercise the ceil-div loop bounds and PSUM placement without ever hitting the mask path—they are the oracle for any change to the scf.for emission. The partial-M, partial-K, and partial-N rows each isolate one axis of the per-element mask, so an off-by-one along any single axis surfaces on its own row rather than mixed with the other two. The two degenerate rows stress the opposite extreme: the ceil-div bounds still round up to one full $128 \times 128 \times 512$ tile even when the live region is a single scalar or a column, so every element except the live region is masked away and the check catches any leak of out-of-range operands into the PSUM accumulator. The tolerance atol = 10^{-4} , rtol = 10^{-2} matches what the nki-samples fp32 matmul tutorials use for their own cross-checks against torch.matmul, so the emitted kernel is held to the same standard as the hand-tuned reference.

7.5 RQ3: structural comparison with nki-samples

Table 3 summarises the key structural differences. The generated kernel mirrors the reference’s loop structure and PSUM placement—the two pieces that the `nki-samples` author calls out explicitly as the “tricky parts” of the kernel—but replaces `nisa.dma_copy` with the higher-level masked `nl.load/nl.store` pair. The mask-every-load pattern is non-idiomatic for an expert human but matches what Triton’s mask semantics actually require: falling back to the `nki-samples` pattern would need a whole separate masking front-end.

The `matmul+ReLU` case is structurally *better* than the unfused hand-written reference. The reference does the `matmul`, materialises the result to HBM, and then runs a standalone `ReLU` kernel that reloads, clamps, and re-stores. Our `nki-fuse-activation` pass folds the `ReLU` into the PSUM—`→SBUF copy` (printing `nisa.activation(op=nl.relu, ...)` in the emitted Python), saving exactly one HBM round-trip per output tile. On an NC-v2 where HBM bandwidth is the usual bottleneck this is a meaningful win; the `matmul` case itself is benchmarked on hardware in the next subsection.

7.6 Wall-clock performance

We benchmark the emitted `matmul` kernel on a Trainium (NC-v2) instance against three hand-written reference kernels using `neuronxcc.nki.benchmark`, which compiles each kernel to a NEFF once and times pure on-device execution (warmup = 5, iters = 10, p50). All timings are in milliseconds per iteration.

Reference choice. We initially targeted the four tutorial kernels in the `nki-samples` `matmul` tutorial (`tilled`, `hoist_load`, `block_free_dim`, `fully_optimized`). Those kernels are written against a newer top-level `nki.* API—nc_matmul` with keyword arguments, Python-list tile storage, a different memset signature—that is not compatible with the `neuronxcc.nki.*` layer `nki.benchmark` drives in our `venv`. Rather than alias the two, we use (i) the sophisticated block-tilled kernel from `nki-samples/contributed/matmul.py` as `fully_opt` (its API matches ours natively), and (ii) two short reference kernels written inline in the same API—one `tilled` (a direct three-loop nest with a fresh `lhsT / rhs` load per (m, n)) and one `hoist_load` (the same kernel with the `lhsT` load hoisted out of the n -loop). This mirrors the optimisation levels of the tutorial kernels at the API layer this `venv` supports.

Table 4 collects the results. Three patterns stand out. **The emitted kernel is $\sim 1.5\text{--}1.7\times$ slower than the simplest reference** across every shape ≥ 2048 . The gap widens roughly linearly with problem size, which rules out a constant-overhead explanation (NRT dispatch, compile time, first-tile latency)

shape ($M \times K \times N$)	emitted	tilled	hoist_load	fully_opt
512 $\times$ 512 $\times$ 512	0.03	0.03	0.03	n/a
2048 $\times$ 1024 $\times$ 2048	0.56	0.47	0.47	0.47
2048 $\times$ 2048 $\times$ 2048	1.44	0.91	0.91	0.89
4096 $\times$ 2048 $\times$ 2048	2.85	1.77	1.77	1.75
4096 $\times$ 4096 $\times$ 4096	11.95	7.89	7.88	6.99

Table 4: p50 on-device latency (ms/iter) via `nki.benchmark`. `fully_opt` requires $K\%1024$, $M\%512$, $N\%2048$ and is reported n/a for the 512³ shape.

and points at the per-tile cost: the most likely culprits are an extra PSUM $\rightarrow$ SBUF copy in the emitted path (the references fuse the cast into the store whereas we emit an explicit `nisa.tensor_copy`) and the load-side masking, which costs non-trivial SBUF lanes even when every mask bit is true. A tile-size inference pass (§9) would sidestep the second cost entirely on aligned inputs.

tilled and hoist_load are indistinguishable (0.47 vs 0.47, 0.91 vs 0.91, 7.89 vs 7.88). Hand hoisting the `lhsT` load out of the n -loop—an optimisation the `nki-samples` tutorial highlights explicitly—buys nothing on this NeuronCC version: the compiler is already performing the same hoist automatically. This is a useful calibration point. It says that the easy-to-reach optimisation in the textbook “`tilled` vs. `hoist_load`” split has already been absorbed into the stack, so the interesting comparisons are only `emitted` vs. `tilled` (the gap we still pay) and `hoist_load` vs. `fully_opt` (the gap the block-tilled transform buys).

fully_opt pays off only at 4096³ (6.99 vs 7.89—about 12% faster than tiled). At smaller shapes its Z -block-stationary storage and `nl.loop_reduce` machinery aren’t amortised. At 512³ the four columns collapse to 0.03 ms, which is a single 128 $\times$ 128 $\times$ 512 tile plus NRT dispatch floor—there is nothing to optimise.

Throughput in TFLOPS. Converting Table 4 to achieved throughput (2MKN FLOPs divided by p50 latency) makes the gap easier to interpret in absolute terms. At 4096³ the emitted kernel runs at 11.5 TFLOPS while `tilled` and `fully_opt` reach 17.4 and 19.7 TFLOPS respectively; at 2048³ the corresponding numbers are 11.9, 18.9, and 19.3 TFLOPS. Within each column the throughput is effectively flat across the 2k–4k range, which is consistent with the gap being a per-tile cost (paid on every inner-loop iteration) rather than a setup- or reduction-bound overhead. All three columns sit well below NC-v2’s advertised peak, but peak is defined for mixed-precision `fp16-in / fp32-accum`, and we are running `fp32` end-to-end—that ceiling applies uniformly, so the meaningful signal is column-to-column.

The emitted kernel is not yet competitive with hand-written NKI, but its gap-from-tiled is a single structural cost rather than an accumulation of inefficiencies, which makes it a targetable next step. The mask-every-load invariant bought correctness on arbitrary shapes (the basis of the RQ2 results); the next iteration of this project would keep that invariant but drop the redundant PSUM→SBUF copy, which is expected to close most of the 1.6× gap.

7.7 Discussion

A few patterns stand out from the evaluation.

First, the *pass ordering* has load-bearing structural constraints. `-nki-fuse-dma` run before `-nki-canonicalize-pid-loops` erases the minsi bounds-check that the pid canonicaliser uses to locate the M function argument; `-nki-fold-psum-init` run before `-linalg-to-nki` leaves a `linalg.fill` seed that is not yet the input to an `nki.nc_matmul` `iter_args` chain. Each pass therefore documents its pre-conditions on the IR shape it expects, not just the IR it produces, and the regression tests pin the canonical ordering.

Second, the *static-tile / dynamic-mask* split is consistently the right call. Triton’s mask semantics guarantee that any out-of-range element of a tile is a don’t-care; once the passes drop the dynamic subview extents, the emitter has a clean invariant to rely on (every load and every store carries a mask computed from the dynamic M/K/N args). The result is that the emitter does not need to special-case shape alignment at all, which is exactly the simplification that the `nki-samples` tutorial spends two paragraphs apologising for.

Third, the *narrow-matching-plus-graceful-failure* pattern (`-nki-check-unsupported-elementwise`) is invaluable. Before it was added, an unsupported activation body (say, a two-op `linalg.generic`) would survive the fusion pass, survive the translator’s `func.walk`, and eventually crash deep inside the Python emitter’s per-op dispatch with a nondescript error. The check pass folds the diagnostic back to the IR level, where the user can read the op location and the note listing the supported activations.

8 Related Work

Triton compilers. Triton [3] compiles to NVIDIA GPUs through a custom backend. `triton-shared` [4] is an upstream-oriented reimplementation that lowers TTIR to MLIR `linalg`, enabling third-party backends like ours. The Triton-to-NKI leg has no prior public implementation that we are aware of; AWS’s own NKI documentation targets the Python DSL directly.

Tensor-engine DSLs. NKI is one of several recent DSLs that expose partitioned accelerator memories: Google’s Pallas-TPU, NVIDIA’s CUTLASS/CuTe DSL, and AMD’s AITemplate each encode a similar tile-and-PSUM-like-accumulator model with different concrete ops. The common pattern in this line of work is that a *tile-aware* IR with explicit buffer kinds sits below the user-facing DSL. Our `nki` dialect plays the same role.

MLIR linalg-to-target pipelines. The IREE [6] and Torch-MLIR projects have set the dominant design pattern for MLIR backends: a small target-specific dialect plus pattern-rewrite passes that consume a `linalg`-level representation. We adopt that pattern; the distinguishing contribution is the specific match set for the `matmul`-and-`epilogue` shape that `triton-shared` emits, and the pattern-matching Python emitter.

9 Limitations and Future Work

Supported kernel shape. The emitter recognises exactly two skeletons: 3-deep `matmul` (with or without a single pointwise epilogue) and 1-deep vector pointwise. A `matmul` with a non-trivial prologue, a reduction-with-activation, or an attention-style softmax would not be recognised. Extending the emitter is structural but not glamorous: each new skeleton needs its own classifier and its own set of op-emission rules.

Data type and precision. Everything is FP32. Matching the PE array’s canonical FP16-input / FP32-accumulator mode would require a dtype-aware pass at the load side, plus a two-type verifier on `nki.nc_matmul`. A bf16 path is the obvious next step for a workload-relevant benchmark.

Non-rectangular tiles. The NC-v2 tensor engine supports arbitrary $M \leq 128, N \leq 512$ tiles, but the emitter hardcodes (128, 512). Kernels whose outer-dimension bounds are less than 128 are wasteful under the emitter; a tile-size inference pass could shrink the emitted tiles in those cases. This is more useful for small-batch inference workloads than for training.

Autotuning. Triton kernels often declare multiple BLOCK configurations and autotune. We currently flatten every BLOCK to the canonical tile, which is safe but defeats the autotuner’s purpose. Supporting BLOCK-aware emission would let us map each Triton config to a matching NKI tile size at emit time.

Formal verification. The “Fast and Verified” course frame naturally raises the question of whether the pass pipeline is correct. Our only correctness argument is end-to-end numerical agreement with PyTorch across the shape sweep; a stronger guarantee would be a mechanised proof that the

`linalg.matmul + linalg.add` pattern has the same denotation as `nki.nc_matmul`. The shape of such a proof is clear (both ops have standard interpretations in a linear-algebra-over-tiles logic), but building it into an MLIR rewrite system is out of scope for this iteration.

Debugging. Today a mis-match between the expected and observed IR shape manifests as a pass silently skipping. The `-nki-check-unsupported-elementwise` pattern shows the benefit of attaching a diagnostic to *every* pattern that expects a specific shape. Extending this to every pass would substantially improve the “it didn’t lower and I don’t know why” failure mode.

10 Conclusion

We have built an end-to-end compilation pipeline that lowers a Triton kernel, through MLIR and a small custom `nki` dialect, into a `@nki.jit` Python kernel for AWS Trainium. The pipeline consists of six narrow MLIR passes plus a pattern-matching Python emitter, and handles `matmul`, `matmul+ReLU`, and vector pointwise workloads. On an 8-configuration $\times$ 10-shape block sweep the emitted kernels numerically agree with `torch.matmul` at the standard fp32 tolerance, and on a

structural basis the emitted code is a close analogue of the hand-tuned `nki-samples` reference—with the `matmul+ReLU` case strictly better, because the `-nki-fuse-activation` pass eliminates one HBM round-trip per output tile. The same compiler-infrastructure ideas generalise to any accelerator whose Python DSL admits a pattern-match-friendly op subset—which is most of them.

References

- [1] AWS Neuron Documentation. *AWS Trainium Architecture and NKI Programmer’s Guide*. Online.
- [2] AWS Neuron. *nki-samples: tutorials and reference kernels for the Neuron Kernel Interface*. Online.
- [3] Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *MAPL*, 2019.
- [4] Meta Incubator. *triton-shared: a TTIR-to-MLIR-linalg bridge for the Triton compiler*. Online.
- [5] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: scaling compiler infrastructure for domain specific computation. In *CGO*, 2021.
- [6] The IREE Authors. *IREE: an MLIR-based retargetable compiler and run-time for ML workloads*. Online.